

MICROPROCESSOR AND MICRO CONTROLLER

ASSIGNMENT

PWM SIGNAL GENERATOR
WITH CUSTOM DUTY CYCLE

Name: Niswanth.B

Register number: 128004168

Course code: ECE201R01

AIM:

To design and implement a PWM (Pulse Width Modulation) signal generator using the STM32F103C8T6 microcontroller, where the duty cycle is configured by the user through a 4×3 matrix keypad and the output is indicated via an LED, with the entered duty cycle value displayed on a 16×2 LCD screen.

workflow:

1. HARDWARE SETUP

The STM32F103C8T6 (Blue Pill) microcontroller was selected as the primary processing unit. The following peripherals were connected to the board – a 4×3 matrix keypad for user input, a 16×2 LCD display for output visualization, an LED on PA0 driven by PWM, and the onboard LED on PC13 for status indication. Power and ground connections were verified, and a potentiometer was connected to the V0 pin of the LCD for contrast adjustment.

2. PIN CONFIGURATION

The IOC file was configured as follows. PA0 was set as TIM2 Channel 1 for PWM output. PA1 to PA4 were configured as Output Push-Pull for driving the keypad rows. PA5 to PA7 were configured as Input with Pull-up for reading the keypad columns. PA11 and PA12 were set as Output Push-Pull for the LCD RS and Enable pins respectively. PB3 to PB10 were configured as Output Push-Pull for the 8-bit LCD data lines. PC13 was set as Output Push-Pull for the onboard LED. The RW pin of the LCD was hardwired to GND to

3.TIMER CONFIGURATION

Timer 2 was configured in PWM Generation mode on Channel 1. The Prescaler was set to 1999 and the Auto Reload Register (ARR) was set to 35999, resulting in a PWM frequency of 1 Hz. The CCR value was calculated using the formula $CCR = \text{Duty\%} \times 360$, where 360 is derived from $(ARR + 1) / 100 = 36000 / 100$.

4. LCD INITIALIZATION:

The 16×2 LCD was initialized in 8-bit mode by sending the standard HD44780 initialization command sequence — 0x38 three times for 8-bit mode and 2-line configuration, followed by 0x08 for display off, 0x01 for clear display, 0x06 for entry mode set, and 0x0C for display on with cursor off. All commands were sent using direct ODR register manipulation with a left shift of 3 to align the data byte to PB3–PB10.

5. KEYPAD SCANNING:

A software scanning routine was implemented to detect key presses on the 4×3 matrix keypad. Each row was pulled LOW one at a time while the remaining rows were held HIGH. The column pins were then read — a LOW on any column pin indicated a key press at the intersection of that row and column. A debounce delay was added after key release to prevent false triggers. The function blocked execution until a valid key was detected.

6. DUTY CYCLE INPUT:

The program waited in a blocking loop for the user to press two digit keys sequentially. The first key press was stored as the tens digit and the second as the units digit. The duty cycle percentage was computed as $(d1 \times 10) + d2$ and clamped to a maximum of 100. The CCR register of TIM2 was then updated as $CCR = \text{duty} \times 360$ to apply the corresponding PWM signal on PA0.

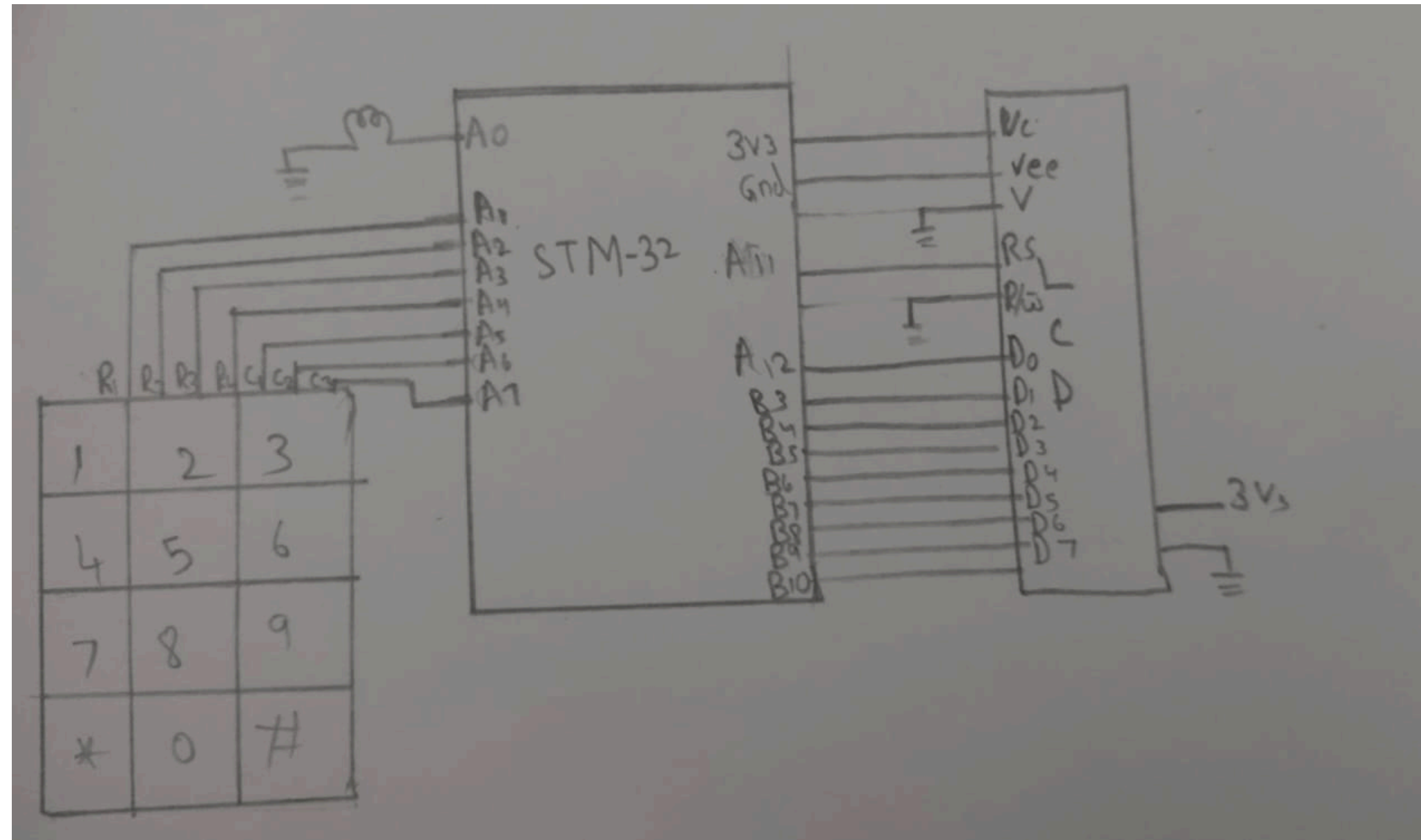
7. LCD DISPLAY

After computing the duty cycle, the LCD was cleared and the cursor was positioned at row 1, column 0 using the 0x80 command. The string "DUTY CYCLE:XX%" was written character by character to the LCD using direct ODR writes with RS set HIGH for data and the Enable pin pulsed for each character.

8. STATUS INDICATION:

The onboard LED on PC13 was used to indicate system status. Since PC13 is active LOW on the Blue Pill, the LED turned on when the pin was driven LOW. The LED blinked once after the first digit was received and once more after the second digit, then stayed on briefly to confirm PWM application before turning off.

interfacing diagram:



code:

```
uint8_t d1, d2, r, c, i;  
uint16_t x1;  
uint16_t row[4] = {GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3, GPIO_PIN_4};  
uint16_t co[3] = {GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7};  
uint8_t b[4][3] = {{1,2,3},{4,5,6},{7,8,9},{0,0,0}};
```

```
uint8_t Keypad_WaitForKey(void)
```

```
{
```

```
    while (1)
```

```
    {
```

```
        for (r = 0; r < 4; r++)
```

```
        {
```

```
            for (i = 0; i < 4; i++)
```

```
                HAL_GPIO_WritePin(GPIOA, row[i], GPIO_PIN_SET);
```

```
            HAL_GPIO_WritePin(GPIOA, row[r], GPIO_PIN_RESET);
```

```
            HAL_Delay(5);
```



```
for (c = 0; c < 3; c++)  
{  
if (HAL_GPIO_ReadPin(GPIOA, co[c]) == GPIO_PIN_RESET)  
{  
while (HAL_GPIO_ReadPin(GPIOA, co[c]) == GPIO_PIN_RESET);  
HAL_Delay(20);  
return b[r][c];  
}}}}}
```

```
HAL_Delay(100);GPIOB->ODR &= ~(0xFF << 3);  
GPIOB->ODR |= (0x38 << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);  
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);
GPIOB->ODR |= (0x38 << 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);
HAL_Delay(10);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);
HAL_Delay(5);
GPIOB->ODR &= ~(0xFF << 3);
GPIOB->ODR |= (0x38 << 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);
HAL_Delay(10);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);  
GPIOB->ODR |= (0x08 << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);  
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);  
GPIOB->ODR |= (0x01 << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);  
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);
GPIOB->ODR |= (0x06 << 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);
HAL_Delay(10);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);
GPIOB->ODR |= (0x0C << 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);
HAL_Delay(10);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);
HAL_Delay(5);
```

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
TIM2->CCR1 = 0;
```

```
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 0);  
d1 = Keypad_WaitForKey();  
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 1);  
HAL_Delay(1000);  
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 0);
```

```
d2 = Keypad_WaitForKey();  
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 1);  
HAL_Delay(1000);  
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 0);
```

```
x1 = (d1 * 10) + d2;  
if (x1 > 100) x1 = 100;  
TIM2->CCR1 = x1 * 360;
```



```
GPIOB->ODR &= ~(0xFF << 3);  
GPIOB->ODR |= (0x01 << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);  
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);  
GPIOB->ODR |= (0x80 << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 0);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);  
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('D' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('U' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('T' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('Y' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= (' ' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('C' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('Y' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('C' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
);
```



```
GPIOB->ODR &= ~(0xFF << 3);
GPIOB->ODR |= ('L' << 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);
HAL_Delay(10);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);
GPIOB->ODR |= ('E' << 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);
HAL_Delay(10);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0);
HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);
GPIOB->ODR |= (':' << 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);
HAL_Delay(10);
```

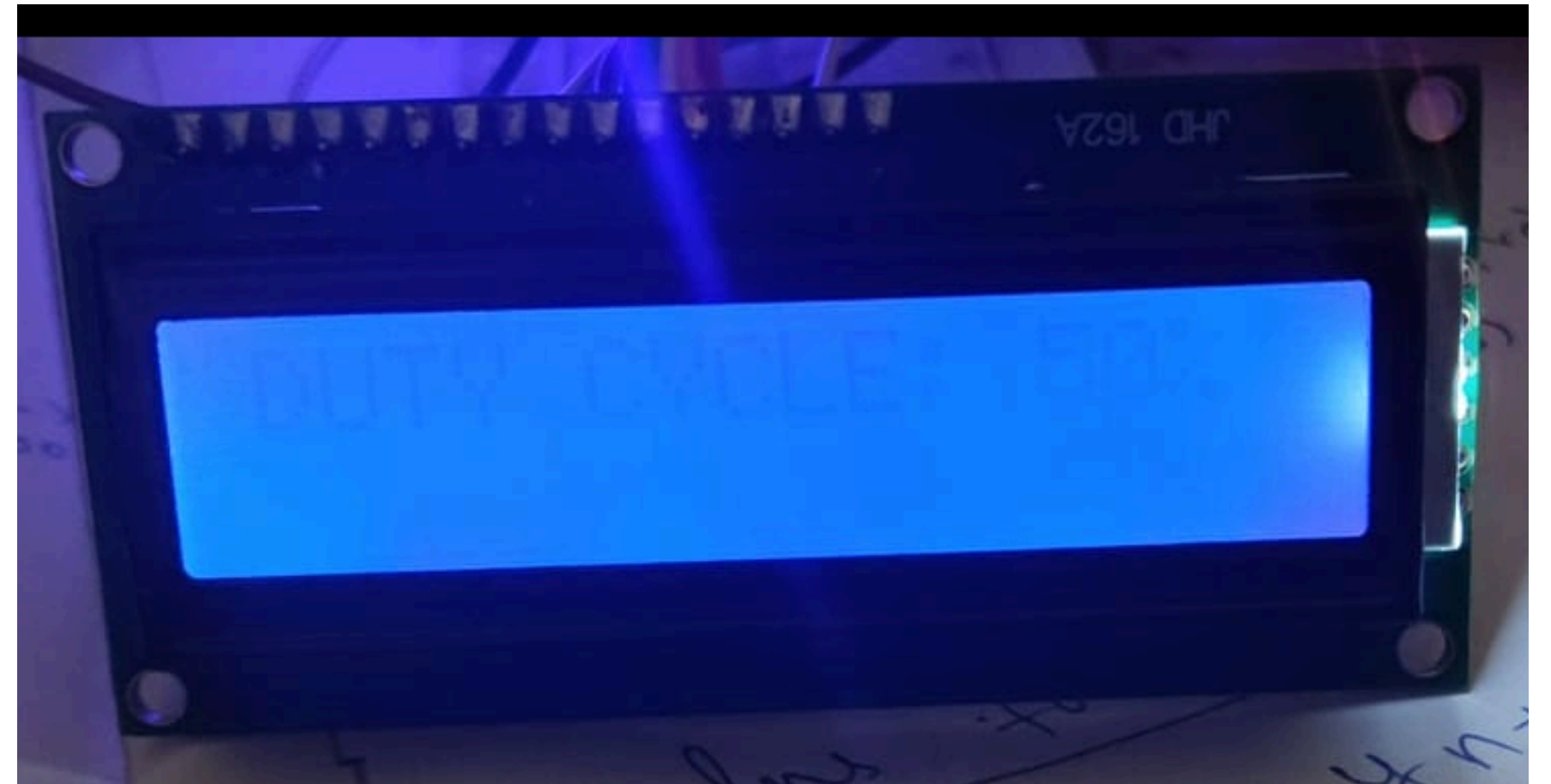
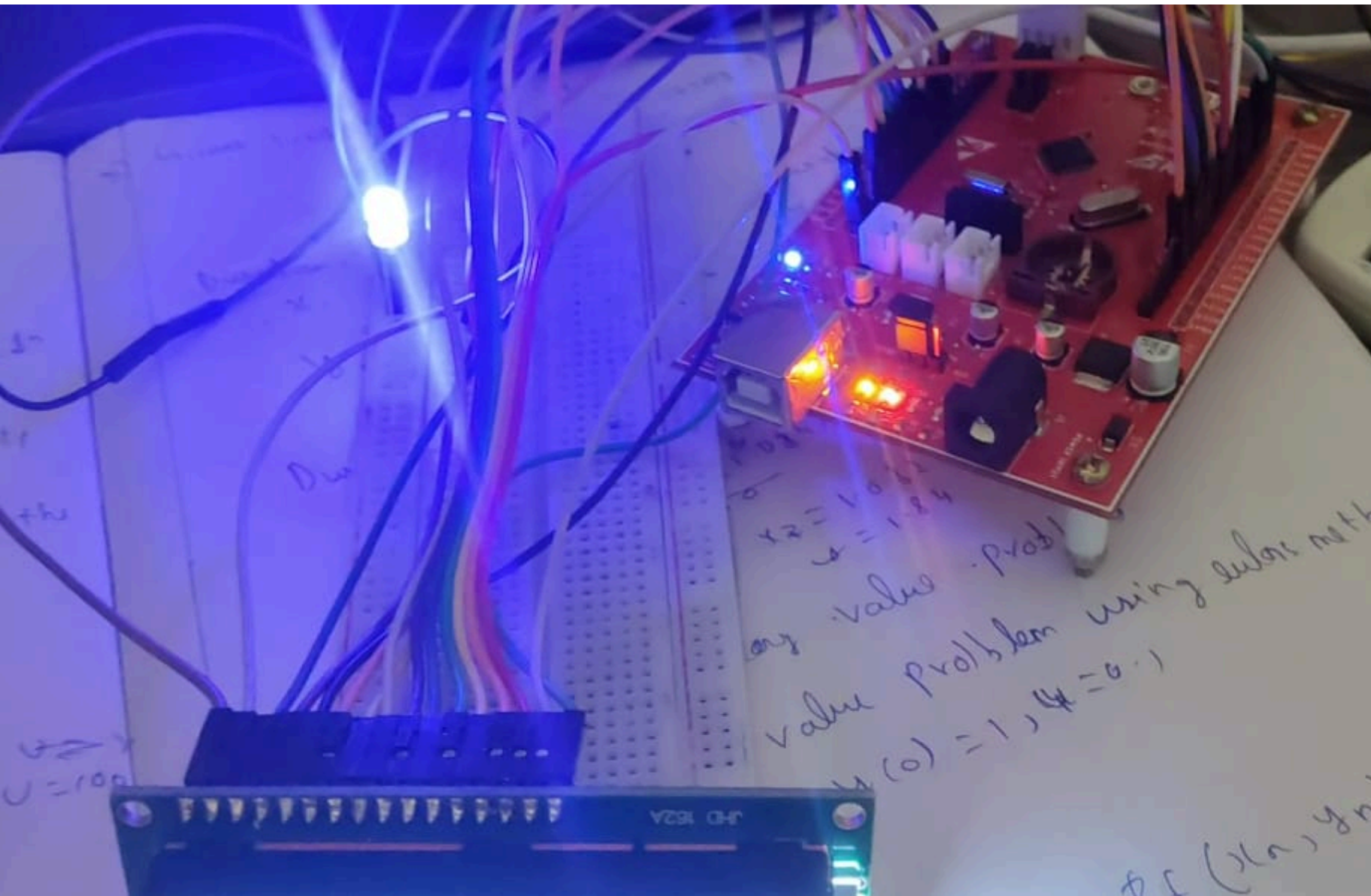
```
GPIOB->ODR &= ~(0xFF << 3);  
GPIOB->ODR |= (('0' + (x1 / 10)) << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3);  
GPIOB->ODR |= (('0' + (x1 % 10)) << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
GPIOB->ODR &= ~(0xFF << 3); GPIOB->ODR |= ('%' << 3);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, 1);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 1);  
HAL_Delay(10); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, 0); HAL_Delay(5);
```

```
HAL_Delay(1000);  
HAL_Delay(5000);
```

OUTPUT:



THANKYOU